

10. Bölüm

Yapılar, Birlikler ve İsim Uzayları

10.1 Yapılar

Yapısal programlamaya adını veren yapı (**structure**), türetilmiş (**derived**) veya kullanıcı tanımlı (**user defined**) bir veri tipidir. Farklı tiplerdeki elemanları bir arada gruplandırarak özel bir veri tipi tanımlamak için **struct** anahtar kelimesini kullanırız. Yapı bir veya birden fazla ilkel veri tipinin (**char**, **int**, **long**, **float**, **double**, ...) ve bu tiplere ait dizilerin bir araya gelmesiyle oluşturulan yeni veri tipleridir.

Diziler, aynı tipte elemanlardan oluşmasına rağmen, yapılar farklı dipte elemanların bir araya gelmesiyle oluşabilir. Aşağıda sözde kodu verilmiştir;

```
struct yapı-kimliği {  
    veri-tipi yapı-elemanı-kimliği1;  
    veri-tipi yapı-elemanı-kimliği1;  
    ...  
};
```

Örnek olarak **adı**, **soyadı**, **numarası**, **yaşı**, **cinsiyeti** gibi farklı öğeler ile bir **Ogrenci** yapısı tanımlanabilir;

```
struct Ogrenci { // C++'ta tür isimlerini büyük harfle başlatmak yaygındır  
    std::string adı;  
    std::string soyadı;  
    int yas;  
    int cinsiyet; // 1: Erkek, 2: Kadın  
};
```

Yapı tanımlandıktan sonra aşağıdaki sözde kodda verildiği gibi yeni değişkenler tanımlanabilir;

```
yapı-kimliği değişken-kimliği[,değişken-kimliği2][,değişken-kimliği3];
```

Yapının elemanlarına tanımlama sırasında **başlangıç değeri** (**initial value**) verilebilir yani yapı değişkeni bazı değerlerle **başlatılabilir** (**initialize**). Bunun için diziler gibi süslü parantez kullanılır;

```
Ogrenci ogrenci1; // Başlatılmamış  
Ogrenci ogrenci2={"Deniz", "AK", 35, 2};  
Ogrenci ogrenci3={"Damla", "YILDIZ", 28, 2};
```

İç İçe Yapılar

Bir yapının içinde bir başka yapı da tanımlanabilir. Yani iç içe yapılar (**nested structs**) tanımlanabilir.

10.2 Yapı Elemanlarına Erişim

Tanımlanan yapı değişkenleri üzerinden her bir alamana **nokta işleci** (**dot operator**) erişilir. Bu işlecin önceliği parantez ile aynıdır. Atama işleci (=) bir yapıyı doğrudan kopyalamak için kullanılabilir. Ayrıca, bir yapının üyesinin değerini başka birine atamak için atama işlecini de kullanabiliriz.

```
#include <iostream>  
#include <string> // string sınıfı için gerekli  
struct Ogrenci { // C++'ta tür isimlerini büyük harfle başlatmak yaygındır  
    std::string adı;  
    std::string soyadı;  
    int yas;  
    int cinsiyet; // 1: Erkek, 2: Kadın  
};  
int main() {  
    // ogrenci1 yapısı başlatılmadığından değerleri aşağıdaki gibi güncelleniyor:  
    // C++ std::string ile doğrudan atama yapabiliriz, strcpy'ye gerek yok.  
    Ogrenci ogrenci1;  
    ogrenci1.adı = "İlhan";
```

```

ogrenci1.soyadi = "OZKAN";
ogrenci1.yas = 50;
ogrenci1.cinsiyet = 1;
std::cout << "Ogrenci 1 Bilgileri:" << std::endl;
std::cout << "Adi: " << ogrenci1.adi << std::endl; // nokta işleci ile adi alanına erişildi.
std::cout << "Soyadi: " << ogrenci1.soyadi << std::endl; // nokta işleci ile soyadi alanına
→ erişildi.
std::cout << "Yasi: " << ogrenci1.yas << std::endl; // nokta işleci ile yas alanına erişildi.
std::cout << "Cinsiyeti: " << ogrenci1.cinsiyet << std::endl; // nokta işleci ile cinsiyet
→ alanına erişildi.
std::cout << "-----" << std::endl;
// yapı bazı değerlerle başlatıldı. 'struct' anahtar kelimesi C++'ta zorunlu değil.
Ogrenci ogrenci2 = {"Deniz", "AK", 35, 2};
std::cout << "Ogrenci 2 Bilgileri:" << std::endl;
std::cout << "Adi: " << ogrenci2.adi << std::endl;
std::cout << "Soyadi: " << ogrenci2.soyadi << std::endl;
std::cout << "Yasi: " << ogrenci2.yas << std::endl;
std::cout << "Cinsiyeti: " << ogrenci2.cinsiyet << std::endl;
std::cout << "-----" << std::endl;
// yapı kopyalama (sığ kopyalama)
Ogrenci ogrenci3 = ogrenci2;
std::cout << "Ogrenci 3 Bilgileri (Ogrenci 2'den kopyalandı):" << std::endl;
std::cout << "Adi: " << ogrenci3.adi << std::endl;
std::cout << "Soyadi: " << ogrenci3.soyadi << std::endl;
std::cout << "Yasi: " << ogrenci3.yas << std::endl;
std::cout << "Cinsiyeti: " << ogrenci3.cinsiyet << std::endl;
}

```

10.3 Yapı Göstericileri

Yapı göstericileri, tıpkı diğer değişkenlere gösterici tanımladığımız gibi tanımlayabiliriz. Gösterici üzerinden yapı değişkenlerine **dolaylı işleç** (**indirection operator**) yani (**->**) ile erişilir. Bu nokta işleci ile aynı önceliklidir.

Yapılar ve göstericileri; veri tabanları, dosya yönetim uygulamaları ve ağaç ve bağlı listeler gibi karmaşık veri yapılarını işlemek gibi farklı uygulamalarda kullanılır.

```

#include <iostream> // std::cout için
#include <string> // std::string için
// C++'ta struct isimlerini Büyük Harfle başlatmak yaygındır
struct Ogrenci {
    std::string adi;
    std::string soyadi;
    int yas;
    int cinsiyet; // 1: Erkek, 2: Kadın
};
int main() {
    Ogrenci ogrenci1;
    // Alanlar std::string tanımlandığından strcpy() yerine doğrudan atama yapabiliriz. Çok daha
    → güvenli.
    ogrenci1.adi = "Ilhan";
    ogrenci1.soyadi = "OZKAN";
    ogrenci1.yas = 50;
    ogrenci1.cinsiyet = 1;
    // İşaretçi tanımlı. Yine 'struct' kelimesi yok.
    Ogrenci* ogrenciGosterici;
    ogrenciGosterici = &ogrenci1;
    // nokta işleci yerine '->' işleci kullanılır.
    std::cout << "Adi: " << ogrenciGosterici->adi << std::endl
    << "Soyad: " << ogrenciGosterici->soyadi << std::endl
    << "Yasi: " << ogrenciGosterici->yas << std::endl
    << "Cinsiyeti: " << ogrenciGosterici->cinsiyet << std::endl;
}

```

10.4 Yapılar ve Fonksiyonlar

Sıradan dizilere benzer şekilde bir yapı dizisi tanımlayabiliriz. Ayrıca yapı değişkenini bir fonksiyona parametre olarak gönderebilir ve bir fonksiyondan bir yapı döndürebilirsiniz;

```
#include <iostream>
#include <string> // std::string sınıfı için gerekli
struct Ogrenci {
    std::string adi;
    std::string soyadi;
    int yas;
    int cinsiyet; // int yerine ileride göreceğimiz doğrudan enum tipi kullanmak daha güvenli
};
Ogrenci yeniOgrenci();
void ogrenciYaz(Ogrenci);
int main() {
    Ogrenci ogrenciler[30]; // 30 elemanlı Ogrenci dizisi
    Ogrenci o = yeniOgrenci();
    ogrenciYaz(o);
}
Ogrenci yeniOgrenci() {
    // std::string ile doğrudan atama yapabiliriz. ERKEK enum değeri tam sayı gibi davranabilir.
    Ogrenci yeni = {"Ilhan", "Ozkan", 50, 2};
    return yeni;
}
void ogrenciYaz(Ogrenci pOgrenci) {
    std::cout << "Ogrenci Bilgileri:" << std::endl;
    std::cout << "Adi: " << pOgrenci.adi << std::endl;
    std::cout << "Soyadi: " << pOgrenci.soyadi << std::endl;
    std::cout << "Yasi: " << pOgrenci.yas << std::endl;
    std::cout << "Cinsiyeti: " << pOgrenci.cinsiyet << " (0:Belirtilmemis, 1:Kadin, 2:Erkek)" <<
        << std::endl;
    std::cout << "-----" << std::endl;
}
```

Yapı elemanları değer tipler olduğundan, yapılara ait referansları parametre olarak kullanmak, olabilecek değişiklikleri aktarmak için üstünlük sağlar. Gösterici kullanımı tehlikelidir;

```
#include <iostream>
#include <string> // std::string sınıfı için gerekli
struct Ogrenci {
    std::string adi;
    std::string soyadi;
    int yas;
    int cinsiyet;
};
Ogrenci yeniOgrenci();
void ogrenciOku(Ogrenci& pOgrenci);
void ogrenciYaz(const Ogrenci& pOgrenci);
int main() {
    Ogrenci o = yeniOgrenci();
    ogrenciYaz(o);
    ogrenciOku(o);
    ogrenciYaz(o);
}
Ogrenci yeniOgrenci() {
    Ogrenci yeni = {"Ilhan", "Ozkan", 50, 1};
    return yeni;
}
void ogrenciOku(Ogrenci& pOgrenci) {
    std::cout << "Adi Giriniz: ";
    std::cin >> pOgrenci.adi; //referans ile de üyerlere nokta işleci ile erişilir.
    std::cout << "Soyadi Giriniz: ";
    std::cin >> pOgrenci.soyadi;
```

```

std::cout << "Yas Giriniz: ";
std::cin >> pOgrenci.yas;
std::cout << "Cinsiyet Giriniz (0:Belirtilmemis, 1:Erkek, 2:Kadin): ";
std::cin >> pOgrenci.cinsiyet;
}
// 'const Ogrenci*' yapının içeriğinin değiştirilmesini engeller (güvenli).
void ogrenciYaz(const Ogrenci& pOgrenci) {
    std::cout << "-----" << std::endl;
    std::cout << "Ogrenci Bilgileri:" << std::endl;
    std::cout << "Adi: " << pOgrenci.adi << std::endl;
    std::cout << "Soyadi: " << pOgrenci.soyadi << std::endl;
    std::cout << "Yasi: " << pOgrenci.yas << std::endl;
    std::cout << "Cinsiyeti: " << pOgrenci.cinsiyet << std::endl;
    std::cout << "-----" << std::endl;
}

```

10.5 İç İç Yapılar

İç içe yapılara (**nested structs**) ait **Ogrenci** yapımıza doğum tarihini içerecek yapı eklenen örnek, aşağıda verilmiştir;

```

#include <iostream>
#include <string> // std::string için gerekli
// C++'ta yapı isimlerini Büyük Harfle başlatmak yaygındır (Ogrenci)
struct Ogrenci {
    std::string adi;
    std::string soyadi;
    int yas;
    int cinsiyet; // 1:erkek, 2:Kadın, 0:Belirtmiyor
    // İç yapıyı isimlendirmek ve bir üye olarak tanımlamak modern C++'ta daha temiz ve
    // → güvenlidir.
    struct Tarih {
        int gun;
        int ay;
        int yil;
    } dogumTarihi;
};
Ogrenci yeniOgrenci();
void ogrenciYaz(Ogrenci);
int main() {
    Ogrenci o = yeniOgrenci();
    ogrenciYaz(o);
}
Ogrenci yeniOgrenci() {
    Ogrenci yeni = {"Ilhan", "Ozkan", 50, 1, {1, 1, 1970}};
    return yeni;
}
void ogrenciYaz(Ogrenci pOgrenci) {
    std::cout << pOgrenci.adi << std::endl;
    std::cout << pOgrenci.soyadi << std::endl;
    std::cout << "Yas:" << pOgrenci.yas << std::endl;
    std::cout << "Cinsiyet:" << pOgrenci.cinsiyet << std::endl;
    // İsimlendirilmiş iç yapı kullandığımız için erişim 'dogumTarihi' üyesi üzerinden yapılır.
    std::cout << "Dogum Tarihi:" << pOgrenci.dogumTarihi.gun << "-" << pOgrenci.dogumTarihi.ay <<
    // → "-" << pOgrenci.dogumTarihi.yil << std::endl;
}

```

10.6 Öz Referanslı Yapılar

Öz referanslı yapı (**self-referential struct**), öğelerinden bir veya daha fazlası kendi tipindeki göstericilerden oluşan bir yapıdır. Öz referanslı yapılar, bağlantılı listeler ve ağaçlar gibi karmaşık olan **dinamik veri yapıları** (**dynamic data structure**) içinde yaygın olarak kullanılırlar.

```

#include <iostream> // std::cout için

```

```
#include <string>    // std::string için
struct Ogrenci {
    std::string adi;
    std::string soyadi;
    int yas;
    Ogrenci* sonrakiOgrenci; // Kendi veri tipinden bir yapı göstericisi
};
// 'const Ogrenci*' yapının içeriğinin değiştirilmesini engeller (güvenli).
void ogrencileriYaz(const Ogrenci*);
int main() {
    // std::string ve nullptr kullanımı ile temiz bir başlatma
    // İlk düğüm: Ilhan
    Ogrenci ilk = {"Ilhan", "OZKAN", 50, nullptr};
    // İkinci düğüm: Ayse
    Ogrenci sonraki = {"Ayse", "YILMAZ", 40, nullptr};
    // Bağlantı: Ilhan'dan Ayse'ye
    ilk.sonrakiOgrenci = &sonraki;
    // Üçüncü düğüm: Tahsin
    Ogrenci birsonraki = {"Tahsin", "BULUT", 35, nullptr};
    // Bağlantı: Ayse'den Tahsin'e
    sonraki.sonrakiOgrenci = &birsonraki;
    // Listenin başını yazdır fonksiyonuna gönderiyoruz
    ogrencileriYaz(&ilk);
}
// 'const Ogrenci*' yapının içeriğinin değiştirilmesini engeller (güvenli).
void ogrencileriYaz(const Ogrenci* pIlk) {
    int sayac = 0;
    while (pIlk != nullptr) {
        std::cout << "OGRENCI " << ++sayac << ": "
        << "Adi: " << pIlk->adi << " "
        << "Soyad: " << pIlk->soyadi << " "
        << "Yas: " << pIlk->yas << std::endl;

        // Bir sonraki düğüme geçiş
        pIlk = pIlk->sonrakiOgrenci;
    }
}
```

10.7 Hizalama

C++'ta doğrudan dilin bir parçası olan `alignof` anahtar kelimesi hizalama için kullanılır. Bir veri tipinin bellekte hangi bayt sınırlarına göre hizalanması (`alignment`) gerektiğini söyler. Bilgisayarın belleğini 4 baytlık (32-bit bir sistemde) kutular olarak düşün. `int` verisi her zaman 4'ün katı olan bir adreste başlamak ister. İşlemci mimarisine göre veriler bellekten genellikle 4 veya 8 baytlık bloklar halinde okunduğundan hizalama buna göre yapılır.

```
#include <iostream>
#include <cstdint> // size_t için

// Örnek bir yapı
struct Yapi {
    char c;    // 1 bayt
    double d;  // 8 bayt
    int i;     // 4 bayt
};

int main() {
    std::cout << "--- Veri Tipleri Hizalama Gereksinimleri ---\n";

    // C++'ta alignof bir anahtar kelimedir, kütüphane gerektirmez
    std::cout << "char hizalamasi    : " << alignof(char) << " bayt\n";
    std::cout << "int hizalamasi     : " << alignof(int) << " bayt\n";
    std::cout << "double hizalamasi  : " << alignof(double) << " bayt\n";
}
```

```

// Yapı hizalaması
std::cout << "struct hizalamasi : " << alignof(Yapi) << " bayt\n";

std::cout << "--- Karsilastirma (Size vs Align) ---\n";
std::cout << "struct boyutu (sizeof) : " << sizeof(Yapi) << " bayt\n";
std::cout << "struct hizalamasi (align): " << alignof(Yapi) << " bayt\n";

return 0;
}
/*Program Çıktısı
--- Veri Tipleri Hizalama Gereksinimleri ---
char hizalamasi : 1 bayt
int hizalamasi : 4 bayt
double hizalamasi : 8 bayt
struct hizalamasi : 8 bayt
--- Karsilastirma (Size vs Align) ---
struct boyutu (sizeof) : 24 bayt
struct hizalamasi (align): 8 bayt
*/

```

§10.7.1 Hizalamayı Zorlamak

C++'ta bir verinin hizalamasını manuel olarak artırmak için `alignas` anahtar kelimesini kullanabiliriz. Bu, özellikle SIMD komut setleri veya ara bellek (`cache`) optimizasyonu için çok kritiktir.

```

struct alignas(16) ÖzelHizalama {
    int x;
};
// sizeof(ÖzelHizalama) artık en az 16 bayt dönecektir.

```

10.8 Yapı Dolgusu

C++ derleyicileri, işlemcinin veriye en hızlı şekilde erişebilmesi için nesneler arasına boşluklar (`padding`) yerleştirir. İşlemci mimarisine göre veriler bellekten genellikle 4 veya 8 baytlık bloklar halinde okunduğundan hizalama buna göre hizalanır.

```

#include <iostream>

// Senaryo 1: Verimsiz (Kötü) dizilim
struct Kotu {
    char a;    // 1 bayt + 3 bayt DOLGU
    int b;     // 4 bayt
    char c;    // 1 bayt + 3 bayt DOLGU
}; // Toplam: 12 bayt

// Senaryo 2: Verimli (İyi) dizilim
struct İyi {
    int b;     // 4 bayt
    char a;    // 1 bayt
    char c;    // 1 bayt + 2 bayt DOLGU
}; // Toplam: 8 bayt

int main() {
    std::cout << "Kotu struct boyutu: " << sizeof(Kotu) << " bayt\n";
    std::cout << "Iyi struct boyutu : " << sizeof(Iyi) << " bayt\n";
    return 0;
}
/*Program Çıktısı
Kotu struct boyutu: 12 bayt
Iyi struct boyutu : 8 bayt
*/

```

İşlemciler (CPU), veriye hizalanmış adreslerden (örneğin 4'ün katı adresler) çok daha hızlı erişir. Eğer bir veri yanlış hizalanırsa, CPU o veriyi okumak için iki kat daha fazla çaba sarf edebilir. Bir yapının hiza-

lama gereksinimi, genellikle içindeki en geniş hizalama gerektiren elemana göre belirlenir. Yukarıdaki örnekte `double` (8 bayt) olduğu için tüm yapı muhtemelen 8 baytlık sınıra hizalanacaktır.

`struct Kotu` için hesaplama:

1. `char a`, 0. adrese yerleşir.
2. Sıradaki `int b`, 4. adreste başlamak zorundadır. Bu yüzden 1, 2 ve 3. adresler boş bırakılır (**padding**).
3. `int b`, 4-7 adreslerini kaplar.
4. `char c`, 8. adrese yerleşir.
5. Yapının toplam boyutu, en büyük elemanın (`int`) hizalamasının katı olmalıdır. Bu yüzden sonuna 3 bayt daha eklenir.
6. Bu durumda

$$\text{Toplam Boyut} = 1(\text{char}) + 3(\text{pad}) + 4(\text{int}) + 1(\text{char}) + 3(\text{pad}) = 12 \text{ bayt}$$

olur

C++ dilinde **yapı dolgusu** (**structure padding**), işlemci (CPU) mimarisi ile belirlenir. Dolgu işlemi ile üyelerinin bellekte doğal olarak hizalanması için belirli sayıda boş bayt eklenir. Bunun nedeni 32 veya 64 bitlik bir bilgisayarda işlemcinin tek seferde bellekten 4 bayt okumasından kaynaklanmaktadır.

`struct Iyi` için hesaplama:

- ◇ `int b`, 0-3 adreslerini kaplar.
- ◇ `char a`, 4. adrese yerleşir.
- ◇ `char c`, 5. adrese yerleşir. Toplam boyutu 4'ün katına tamamlamak için sona sadece 2 bayt dolgu eklenir.
- ◇ Bu durumda

$$\text{Toplam Boyut} = 4(\text{int}) + 1(\text{char}) + 1(\text{char}) + 2(\text{pad}) = 8 \text{ bayt}$$

olur.

Hizalama ve Dolgu

`alignof` bize bir tipin "hizalama kuralını" söylerken, bu kurala uymak için derleyicinin aralara attığı boşluklara dolgu (**padding**) diyoruz. **Büyük sistemlerde veya gömülü cihazlarda**, değişkenleri büyükten küçüğe doğru (örneğin önce `double`, sonra `int`, en son `char`) sıralamak bellekten **ciddi tasarruf** sağlar.

§10.8.1 Dolguyu Engelleme

Bazı durumlarda (ağ üzerinden veri paketi gönderirken veya dosya formatı okurken) derleyicinin boşluk eklemesini istemeyiz. C++ standartlarında doğrudan bir "`packed`" anahtar kelimesi olmasa da, derleyicilerin sunduğu iki yaygın yöntem vardır:

1. Derleyiciye tüm veriler için hizalama sınırını 1 bayt yapmasını söylemek için `#pragma pack` ön işlemci yönergesi koda eklenir. Sadece belirlenen veri için hizalama sınırını 1 bayt yapmasını istemek için
2. C++11 ile birlikte gelen `[[gnu::packed]]` özniteliği kullanılır.

```
#include <iostream>
```

```
#pragma pack(push, 1) // Hizalamayı 1 bayt yap ve eski ayarı sakla
```

```
struct Paket1 {
```

```
    char a;
```

```
    int b;
```

```
    char c;
```

```
};
```

```
#pragma pack(pop) // Eski hizalama ayarlarına geri dön
```

```
// GCC/Clang için modern alternatif
```

```
struct [[gnu::packed]] Paket2 {
```

```
    char a;
```

```
    int b;
```

```
    char c;
```

```
};
```

```
int main() {
```

```
    std::cout << "Standart int boyutu: " << sizeof(int) << "\n";
```

```

std::cout << "Paket1 (Packed) boyutu: " << sizeof(Paket1) << "\n"; // 6 bayt
std::cout << "Paket2 (Packed) boyutu: " << sizeof(Paket2) << "\n"; // 6 bayt
return 0;
}
/*Program Çıktısı
Standart int boyutu: 4
Paket1 (Packed) boyutu: 6
Paket2 (Packed) boyutu: 6
*/

```

Dolgu İşlemi

Dolguyu (**padding**) kapatmak (**packing**), bellekten tasarruf sağlasa da işlemcinin veriye erişimini yavaşlatabilir. Ayrıca bazı işlemci mimarilerinde (örneğin eski ARM veya bazı RISC mimarileri) hizalanmamış bellek erişimi programın çökmesine (**bus error**) neden olabilir. Bu nedenle dolguyu kapatma işlemini sadece veri transferi gibi zorunlu durumlarda kullanılmalıdır.

10.9 Yapılarda Bit Alanları

Bit alanları (**bit field**), boyutu azaltmak için C ve C++ dili yapılarını (**struct**) sıkıca paketler. Yapı üyeleri için bit sayısı verilir ve derleyici bit düzeyinde alanları uyarlar. Bu nedenle bit alan üyesinin adresini alınamaz! `sizeof()` kullanımına da izin verilmez!

```

struct Date
{
    unsigned int Year : 13; // 2^13 = 8192, Yıl değerini tutmak için yeterlidir
    unsigned int Month: 4;  // 2^4 = 16, Ay değerini tutmak için yeterlidir.
    unsigned int Day:   5;  // 2^5 = 32, Gün değerini tutmak için yeterlidir
};

```

Tüm yapı sadece 22 bit kullanıyor ve normal derleyici ayarlarıyla bu yapının boyutu 4 bayt olur. Kullanımı oldukça basittir. Sadece değişkeni bildirimi yapılır ve sıradan bir yapı gibi kullanılır;

```

#include <iostream>
struct Date
{
    unsigned int Year : 13; // 2^13 = 8192, Yıl değerini tutmak için yeterlidir
    unsigned int Month: 4;  // 2^4 = 16, Ay değerini tutmak için yeterlidir.
    unsigned int Day:   5;  // 2^5 = 32, Gün değerini tutmak için yeterlidir
};
int main() {
    Date d;
    d.Year = 2025;
    d.Month = 2;
    d.Day = 28;
    std::cout << "Year:" << d.Year << std::endl //2025
    << "Month:" << d.Month << std::endl //2
    << "Day:" << d.Day << std::endl; //28
    std::cout << sizeof d << std::endl; // 4
    //std::cout << sizeof d.Year << std::endl; HATA Olur!
}

```

10.10 Birlikler

Birlikler (**union**), yapılar (**struct**) gibi tanımlanır. Aralarındaki fark **yapı elemanlarının her birine ayrı bellek bölgesi ayrılırken, birlik üyelerinin her biri aynı bellek bölgesini paylaşırlar**. Birlik tanımına ilişkin sözde kod aşağıda verilmiştir;

```

union yapı-kimliği {
    veri-tipi yapı-elemanı-kimliği1;
    veri-tipi yapı-elemanı-kimliği1;
    ...
};

```


Birliğin Bellek Boyutu

Birliğin bellek boyutu, en fazla bellek kaplayan elemanı kadardır.

Aşağıda üyelerinin aynı bellek bölgesini paylaştığını gösteren program örneği bulunmaktadır;

```
#include <iostream>
#include <iomanip>
union Birlik { // C++'ta union isimlerini Büyük Harfle başlatmak yaygındır
    char baytlar[4];
    int tamsayi;
    char bayt;
    short int kisatamsayi;
};
int main() {
    Birlik b;
    // Boyut kontrolleri: birlik üyeleri kaç bayt?
    std::cout << "sizeof Birlik.baytlar: " << sizeof(b.baytlar) << std::endl;
    std::cout << "sizeof Birlik.bayt: " << sizeof(b.bayt) << std::endl;
    std::cout << "sizeof Birlik.tamsayi: " << sizeof(b.tamsayi) << std::endl;
    std::cout << "sizeof Birlik.kisatamsayi: " << sizeof(b.kisatamsayi) << std::endl;
    std::cout << "sizeof Birlik: " << sizeof(Birlik) << std::endl;
    b.tamsayi = 0x1B2C3D4F; // b'in tam sayı üyesi değişiyor. Bakalım diğer üyeler ne oldu?
    std::cout << "-----" << std::endl;
    std::cout << "b.tamsayi: 0x" << std::hex << std::setw(8) << std::setfill('0') << b.tamsayi <<
        << std::dec << std::endl;
    // char türlerini hex olarak yazdırmak için (int) cast'i gereklidir, aksi takdirde karakter
    // olarak yazdırılırlar.
    std::cout << "b.baytlar: " << std::hex << std::setw(2) << std::setfill('0') << (int)(unsigned
        << char)b.baytlar[0] << "-" << std::hex << std::setw(2) << std::setfill('0') <<
        << (int)(unsigned char)b.baytlar[1] << "-" << std::hex << std::setw(2) << std::setfill('0')
        << << (int)(unsigned char)b.baytlar[2] << "-" << std::hex << std::setw(2) <<
        << std::setfill('0') << (int)(unsigned char)b.baytlar[3] << std::dec << std::endl;
    std::cout << "b.bayt: 0x" << std::hex << std::setw(2) << std::setfill('0') << (int)(unsigned
        << char)b.bayt << std::dec << std::endl;
    std::cout << "b.kisatamsayi: 0x" << std::hex << std::setw(4) << std::setfill('0') <<
        << b.kisatamsayi << std::dec << std::endl;
    std::cout << "-----" << std::endl;
    // Üyelerden biri değiştirildiğinde diğer tüm üyelerin içeriği değişir!
    b.bayt = 0x11;
    std::cout << "Değiştikten Sonra: b.tamsayi: 0x" << std::hex << std::setw(8) <<
        << std::setfill('0') << b.tamsayi << std::dec << std::endl;
}
/*Program Çıktısı:
sizeof Birlik.baytlar: 4
sizeof Birlik.bayt: 1
sizeof Birlik.tamsayi: 4
sizeof Birlik.kisatamsayi: 2
sizeof Birlik: 4
-----
b.tamsayi: 0x1b2c3d4f
b.baytlar: 4f-3d-2c-1b
b.bayt: 0x4f
b.kisatamsayi: 0x3d4f
-----
Değiştikten Sonra: b.tamsayi: 0x1b2c3d11
*/
```

10.11 İsim Uzayları

İsim uzayları (**namespace**), değişkenleri (**variable**), sabitler (**constexpr**), fonksiyonları (**function**), yapıları (**struct**), birlikleri (**union**) ve ileride göreceğimiz sınıfları (**class**) bir isim altında toplayabileceğimiz bir tanım-

lamadır. Birden fazla kütüphane kullanırken isim çakışmalarını önlemek için kullanılır. Böylece aynı kimlikli sınıf, değişken ve fonksiyon birden fazla isim uzayında bulunabilir.

Buna bir örnek verelim. `funk()` kimlikli bir fonksiyonu olan bir kod yazıyor olabilirsiniz ve aynı fonksiyon aynı kimlikle başka bir başlıkta (**header**) da mevcut olabilir. Bu durumda derleyicinin, kodunuz içinde hangi `funk()` fonksiyona atıfta bulunduğunuzu bilmesinin bir yolu yoktur. İsim uzayları, bu zorluğun üstesinden gelmek için tasarlanmıştır ve farklı kütüphanelerde bulunan aynı kimliğe sahip benzer fonksiyonları, sınıfları, değişkenleri vb. ayırt etmek için ek bilgi olarak kullanılır. İsim uzaylarına en iyi örnek, C++ standart kütüphanesidir (`std`). Bu nedenle, C++ programı yazarken her seferinde "`std::`" yazmamak için kodun başında `using namespace std;` yönergesini ekleriz;

§10.11.1 İsim Uzayı Tanımlama

Bir isim uzayının tanımı, aşağıdaki gibi `namespace` anahtar sözcüğüyle tanımlanır;

```
namespace isim-uzayı-kimliği
{
    // kod;
    // -değişken tanımlama (int a,b;)
    // -fonksiyon tanımlama (void topla();)
    // -yapı tanımlama (struct Yapi{ /* ... */ };)
    // -birlik tanımlama (union Yapi{ /* ... */ };)
    // -sınıf tanımlama (class Kisi{ /* ... */ };)
}
```

Burada isim uzayına ilişkin bloğun noktalı virgül ile bitmediği gözden kaçırılmamalıdır. Çünkü **isim uzayına ait bloğun görevi tanımlamaları gruplamaktır**. İsim uzayının içindeki bir öğeye erişmek için **kapsam çözümleme işleci** (`scope resolution operator`) kullanılır. Ayrıca `using namespace` yönergesini (**directive**) kullanarak isim uzaylarını ile kapsam çözümleme işlecinin önek olarak eklenmesini de önleyebilirsiniz.

```
#include <iostream>
using namespace std; /* std:: isim uzayını kullanmak için yönerge: Bu yönerge ile "std::cout"
→ gibi talimatlar sadece "cout" şeklinde kullanılabilir. */
namespace birinci // birinci kimlikli isim uzayı
{
    int tamsayi=100; // Birinci isim uzayındaki tamsayi değişkeni
    void fonksiyon() {
        cout << "Birinci isim uzayındaki fonksiyon." << endl;
    }
}
namespace ikinci // ikinci kimlikli isim uzayı
{
    int tamsayi=200; // ikinci isim uzayındaki tamsayi değişkeni
    void fonksiyon() {
        cout << "İkinci isim uzayındaki fonksiyon." << endl;
    }
}
using namespace birinci; /* birinci:: isim uzayını kullanmak için yönerge: Bu yönerge ile
→ "birinci::tamsayi" gibi talimatlar sadece "tamsayi" şeklinde kullanılabilir. */
int main () {
    fonksiyon(); // birinci isim uzayındaki fonksiyon çağrılır.
    cout << tamsayi << endl; // birinci isim uzayındaki tamsayi kullanılır.
    ikinci::fonksiyon(); // kapsam çözümleme işleci ile ikinci isim uzayındaki fonksiyon
→ çağrılır.
    cout << ikinci::tamsayi << endl; // kapsam çözümleme işleci ile ikinci isim uzayındaki
→ tamsayi kullanılır.
}
/* Programın Çıktısı:
Birinci isim uzayındaki fonksiyon.
100
İkinci isim uzayındaki fonksiyon.
200
*/
```

İsim uzayları, aşağıdaki şekilde iç içe (**nested**) de yerleştirilebilir;

```
#include <iostream>
using namespace std; /* std:: isim uzayını kullanmak için yönerge: Bu yönerge ile "std::cout"
→ gibi talimatlar sadece "cout" şeklinde kullanılabilir. */

namespace birinci // birinci kimlikli isim uzayı
{
    int tamsayi=100; // Birinci isim uzayındaki tamsayi değişkeni
    void fonksiyon() {
        cout << "Birinci isim uzayındaki fonksiyon." << endl;
    }
    namespace ikinci // ikinci kimlikli iç isim uzayı
    {
        int tamsayi=200; // ikinci isim uzayındaki tamsayi değişkeni
        void fonksiyon() {
            cout << "İkinci isim uzayındaki fonksiyon." << endl;
        }
    }
}
using namespace birinci::ikinci; /* birinci::ikinci:: iç isim uzayını kullanmak için yönerge */
int main () {
    birinci::fonksiyon(); // birinci isim uzayındaki fonksiyon çağrılır.
    cout << birinci::tamsayi << endl;
    fonksiyon(); // birinci isim uzayı içindeki ikinci isim uzayındaki fonksiyon çağrılır.
    cout << tamsayi << endl;
}
/* Program çalıştığında:
Birinci isim uzayındaki fonksiyon.
100
İkinci isim uzayındaki fonksiyon.
200
*/
```

§10.11.2 Bağımlı Argüman Arama

Açık bir isim uzayı yönergesi olmadan bir fonksiyonu çağırırken, derleyici o işlevin prototipinden biri de o isim uzayında ise, o isim uzayı içindeki bir işlevi çağırılmayı seçebilir. Buna **bağımlı argüman arama** ADL (**argument dependent lookup**) denir;

```
namespace Test
{
    int func(int i);
    struct Yapi {
        //...
    };
    int func2(const Yapi &data);
}
int main() {
    //...
    func(5); /*Hata alınır. Hangi İsim Uzayından olduğu belli değil. Çünkü "int func(int);"
→ şeklilde prototipi olan bir sürü fonksiyon vardır. */

    Test::Yapi data;
    func2(data); /* Hata alınmaz. Kullandığı argümana göre isim uzayı belli. Çünkü "int
→ func(const Yapi &);" şeklilde prototipi olan tek bir fonksiyon vardır. */
}
```

İsim Uzayını Kullanmayı Alışkanlık Haline Getirmek

En uygun kullanım, isim uzayı adı ile kapsam çözümleme işlecini birlikte kullanmaktır.

§10.11.3 İsim Uzaylarını Genişletme

İsim alanlarının kullanışlı bir özelliği de onları genişletebilmemizdir. Daha sonradan başka bir dosyada bile isim uzayına yeni üyeler ekleyebilirsiniz. Buna örnek olarak `<iostream>` ve `<string>` başlık dosyaları verilebilir. Birinci kaynak dosyamız aşağıdaki gibi olabilir;

```
//TEST1.CPP Dosyası:
namespace Test
{
    void fonk1() {}
}
//arada başka kodlar yer alabilir
namespace Test
{
    void fonk2() {}
}
```

İkinci dosyamız da aşağıdaki gibi olabilir;

```
//TEST2.CPP Dosyası:
namespace Test
{
    void fonk3() {}
}
//arada başka kodlar yer alabilir
namespace Test
{
    void fonk4() {}
}
```

§10.11.4 using Anahtar Kelimesinin İsim Uzaylarında Kullanılması

Bir `using` yönergesi, `namespace` anahtar sözcüğüyle birleştirildiğinde o isim uzayını belirtmeden içindeki kullanırsınız.

```
namespace Test
{
    void func() {}
    void func2() {}
}
int main( {
    //...
    Test::func2(); //kapsam çözümleme işleci kullanarak istediğimiz üyeyi kullanabiliriz
    //...
    using namespace Test; /* Test isim uzayını bu yönerge ile projemize dahil ediyoruz. Artık
    → kapsam çözümleme işlecini kullanmamıza gerek kalmaz. */
    func();
    func2();
    //...
}
```

Tüm isim uzayı yerine, isim uzayındaki seçili üyeleri projeye dahil etmek de mümkündür;

```
namespace Test
{
    void func() {}
    void func2() {}
}
int main( {
    //...
    Test::func2(); //kapsam çözümleme işleci kullanarak istediğimiz üyeyi kullanabiliriz
    //...
    using namespace Test::func; /* Test isim uzayının içindeki func fonksiyonunu bu yönerge ile
    → projemize dahil ediyoruz. Artık bu fonksiyon için kapsam çözümleme işlecini kullanmamıza
    → gerek kalmaz. */
    func(); //Başarılı bir şekilde projeye dahil edilmiştir.
    func2(); //Hata: Bu fonksiyon projeye dahil edilmemiştir.
```

```
//...
}
```

Özellikle başlık (**header**) dosyalarında `using namespace` kullanmak çoğu durumda kötü görülür. Bu yapılırsa, isim uzayı dahil edilmiş başlığı içeren her dosya projeye aktarılır. Bu da çatışmalara yol açabilir. Birinci başlık dosyamız şöyle olsun;

\\BIRINCI.H Başlık Dosyası:

```
namespace AAA
{
    class C;
}
```

İkinci başlık dosyamız da şöyle olsun;

\\IKINCI.H Başlık Dosyası:

```
namespace BBB
{
    class C;
}
```

Birinci başlık dosyasını kullanan bir başka başlık dosyamız aşağıdaki gibi olsun;

//UCUNCU.h Baslik Dosyasi:

```
using namespace AAA;
```

Son olarak bu başlık dosyalarını kullanan ana dosyamız şöyle olsun;

```
//MAIN.CPP Dosyasi:
#include "IKINCI.h"
#include "UCUNCU.h"
using namespace AAA;
int main() {
    //...
    C c; // Hata: BBB::C mi? AAA::C mi?.
    //...
}
```

10.12 Düşün ve Çöz

- ◊ **Düşün:** Yapı Dolgusu kavramını açıklayınız. İşlemcinin veriye erişim hızı ile bellek tasarrufu arasındaki denge bu noktada nasıl kurulur?
- ◊ **Düşün:** Birlik (**union**), neden aynı anda sadece bir üyenin değerini tutabilir? Bellek paylaşımı mantığını bir şema ile açıklayınız.
- ◊ **Düşün:** Bit alanları (**bit field**) kullanımı, düşük seviyeli donanım kontrolünde (**register**) neden vazgeçilmezdir?
- ◊ **Çöz:** Bir öğrencinin adını, numarasını ve üç dersinin notunu tutan bir **struct** tanımlayınız. Bu yapıdan bir dizi oluşturup sınıf ortalamasını hesaplayınız.
- ◊ **Çöz:** İç içe yapılar (**nested structs**) kullanarak bir "Şirket" yapısı ve bu yapının içinde "Departman" ve "Çalışan" bilgilerini tutan bir model oluşturunuz.
- ◊ **Çöz:** Bir **union** kullanarak, 4 baytlık bir **int** sayının bellekteki her bir baytına karakter göstericisiyle erişen bir program yazınız.
- ◊ **Çöz** İsim çakışması (**name collision**) sorunu büyük projelerde neden kritik bir probleme dönüşür? İsim uzaylarının (**namespace**) bu sorunu nasıl çözdüğünü somut bir örnekle gösterin.
- ◊ **Düşün:** `using namespace std;` direktifinin kullanımının neden büyük projelerde sorunlara yol açabileceğini açıklayın. Alternatif olarak ne kullanılması önerilir?
- ◊ **Düşün:** Bağımlı argüman arama ADL (**argument dependent lookup**) nedir? `operator<<` gibi işleçlerin neden doğru ad uzayında bulunabildiğini ADL mekanizmasıyla açıklayın.